

Dato il seguente schema relazionale,

TURISTA(username, cognome, nome, data_nascita, nazionalita)

ATTRAZIONE(codice, nome, descrizione, latitudine, longitudine, via, num_civico, citta, provincia)

RECENSIONE(attrazione, turista, timestamp, titolo, testo, voto)

Chiavi primarie sottolineate e vincoli di integrità:

RECENSIONE.attrazione → ATTRAZIONE,

RECENSIONE.turista → TURISTA,

Formulare in SQL le seguenti interrogazioni, usare la funzione EXTRACT(field FROM source) per estrarre una componente (es. anno) da un timestamp:

i. Trovare tutte le attrazioni che hanno ricevuto nel 2025 un numero di recensioni superiori a quelle ricevute nel 2024. Per ciascuna attrazione riportare: il codice, il nome, la città, il numero di recensioni del 2025.

```
SELECT a.codice, a.nome, a.citta, count(*)
FROM attrazione a JOIN recensione r ON (a.codice = r.attrazione)
WHERE EXTRACT( YEAR FROM r.timestamp ) = '2025'
GROUP BY a.codice
HAVING COUNT(*) > ( SELECT COUNT(*)
                     FROM recensione r1
                     WHERE r1.attrazione = a.codice
                     AND EXTRACT( YEAR FROM r1.timestamp ) = '2024');
```

ii. Trovare tutti i turisti che nel 2025 non hanno mai rilasciato recensioni con un voto > 2 per attrazioni fuori dalla provincia di Verona. Per ciascun turista riportare il nome, il cognome ed il numero di recensioni complessive rilasciate.

```
SELECT t.nome, t.cognome, count(*)
FROM turista t JOIN recensione r ON ( t.username = r.turista )
WHERE t.username NOT IN ( SELECT r1.turista
                          FROM recensione r1 JOIN attrazione a1 on (r1.attrazione = a1.codice)
                          WHERE EXTRACT(YEAR from r1.timestamp) = '2025'
                          AND a1.provincia <> 'Verona'
                          AND r1.voto > 2 )

GROUP BY t.username
```

iii. Trovare tutti i turisti che nel 2025 hanno rilasciato recensioni con un voto medio superiore alla media dei voti massimi rilasciati dagli altri turisti negli anni precedenti. Ritornare lo username dei turisti.

```
CREATE VIEW voti_massimi AS
SELECT MAX(voto) as max
FROM recensione r
WHERE EXTRACT( YEAR FROM timestamp ) < '2025'
GROUP BY r.turista

SELECT r.turista
FROM recensione r
WHERE EXTRACT( YEAR FROM timestamp ) = '2025'
GROUP BY r.turista
HAVING AVG(voto) > (SELECT AVG( max ) FROM voti_massimi )
```

iv. Trovare tutte le attrazioni della città di Verona con nome che inizia con la lettera 'f' maiuscola o minuscola.

```
SELECT *
FROM attrazione
WHERE comune = 'Verona' AND name ilike 'f%'
```

Dato il seguente schema relazionale,

AUTO(targa, num_telaio, modello, colore, km, cilindrata, posti, proprietario)

PRENOTAZIONE(codice, parcheggio, posto, auto, data_prenotazione, data_ora_inizio, data_ora_fine, tipo, pagato)

PARCHEGGIO(nome, comune, indirizzo, coperto)

Chiavi primarie sottolineate e vincoli di integrità:

PRENOTAZIONE.auto → AUTO,

PRENOTAZIONE.parcheggio → PARCHEGGIO,

Formulare in SQL le seguenti interrogazioni:

i. Trovare tutti i parcheggi in cui vengono effettuate più prenotazioni online che prenotazioni fisiche. Per ciascun parcheggio riportare il nome, il comune e l'indirizzo in cui si trova.

```
SELECT nome
FROM parcheggio p JOIN prenotazione r ON ( p.nome = r.parcheggio )
WHERE r.tipo <> 'fisica'
GROUP BY r.parcheggio
HAVING COUNT(*) > ( SELECT count(*)
                     FROM parcheggio p1 JOIN prenotazione r1 ON ( p1.nome = r1.parcheggio )
                     WHERE r1.categoria = 'fisica' AND
                           p1.codice = p.codice )
```

ii. Trovare tutti i parcheggi in cui sono mai state fatte prenotazioni di tipo 'fisica' nel 2024. Per ciascun parcheggio riportare il nome ed il numero delle prenotazioni complessive.

```
SELECT nome, COUNT(*)
FROM parcheggio p JOIN prenotazione r ON ( p.codice = r.parcheggio )
WHERE p.codice NOT IN ( SELECT p1.codice
                       FROM parcheggio p1 JOIN prenotazione r1 ON (p1.codice = r1.parcheggio)
                       WHERE r1.data_ora_prenotazione > '01/01/2024' AND
                             r1.data_ora_prenotazione < '01/01/2024' AND
                             r1.tipo = 'fisica' )

GROUP BY p.codice;
```

iii. Trovare quanti parcheggi del comune di 'Verona' hanno avuto un numero di prenotazioni complessive nel 2024 superiore al numero medio di prenotazioni complessive del 2024 dei parcheggi del comune di 'Padova'.

```
CREATE VIEW nprenotazioni AS
( SELECT r.codice, r.comune, COUNT(*) as num
  FROM prenotazione r join parcheggio p on (r.parcheggio = p.codice)
  WHERE data_prenotazione > '01/01/2024' and data_prenotazione < '01/01/2024'
  GROUP BY r.codice, r.comune
)
```

```
SELECT count(*)
FROM nprenotazioni n
WHERE n.comune = 'Verona'
AND n.num > ( SELECT AVG( n1.num )
              FROM nprenotazioni n1
              WHERE n1.comune = 'Padova' );
```

iv. Trovare tutti i parcheggi coperti del comune di Verona.

```
SELECT *
FROM parcheggio
WHERE comune = 'Verona' AND coperto = 'true';
```